VOL. 25

Flask

Build, test and ship a complete web application with Python and Flask

Source	flask.palletsprojects.com/tutorial
Generated	2026-10-10
Level	Beginner
Estimated study time	20 hours
Language	English
Volume	Vol. 25 - Flask

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 3

Chapter map 3

Chapter 1 - Hello, Flask 4

1.1 Summary 4

1.2 Concepts 5

What a microframework means in practice 5

The request cycle 5

The Python you need before starting 6

1.3 Examples 6

1.4 Common mistakes and best practices 9

1.5 Practice exercises 10

Solutions 10

1.6 Mini test 10

Answer key and explanations 11

Chapter 2 - Project layout and the application factory 11

2.1 Summary 11

2.2 Concepts 12

Why a factory instead of a global app 12

The layout 12

How Flask finds the application 13

2.3 Examples 13

2.4 Common mistakes and best practices 15

2.5 Practice exercises 15

Solutions 15

2.6 Mini test 16

Answer key and explanations 16

How to use this volume

Prerequisites. Python 3.10 or newer (the examples were checked with Python 3.13.15) and comfort with functions, classes, modules, decorators and virtual environments. Basic HTML and SQL are used from Chapter 5 onwards. No previous web framework experience is assumed.

Estimated study time. About 20 hours: one to two hours per chapter, plus the time you spend on the exercises and on your own experiments. Chapters 1 to 8 build the core application; chapters 9 to 11 make it installable, tested and deployed.

How the chapters work. Every chapter opens with a summary, explains the concepts, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and closes with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. The volume builds one small blog application called Blogsmith. Blocks tagged `py` are Python modules, `sql` holds SQLite statements, `html` holds Jinja2 templates, `toml` is package metadata, `bash` and `console` are terminal sessions, and `text` shows output. The pure-Python, Jinja2, SQLite and pytest examples were executed with Python 3.13.15 while this volume was written. The writing machine had no network access and Flask itself was not installed, so every block that depends on Flask is labelled as hand-checked, and its `text` output is the expected result rather than a captured session.

Tip. Create the project once, in Chapter 2, and keep it open beside you. Each chapter adds files to the same application, so the volume ends with one working program that you can extend.

shell

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install "Flask>=3.1,<4" pytest
flask --app blogsmith run --debug
```

output

```
(not executed: Flask could not be installed on the writing machine.
 Expected output once Flask 3.1 is installed: the development server starts,
 prints "Running on http://127.0.0.1:5000", and the application answers there.)
```

Chapter map

#	Chapter	You will learn
1	Hello, Flask	what a microframework is, the WSGI request cycle, the Python you need
2	Project layout and the application factory	directory layout, the factory function, the development server

#	Chapter	You will learn
3	Configuration, static files and the instance folder	config sources, secrets, serving CSS
4	Routing, views and blueprints	routes and converters, `url_for`, organising code with blueprints
5	The database layer with SQLite	connections per request, schema, queries, integrity
6	Registration, login and sessions	password hashing, signed-cookie sessions, `login_required`
7	Templates with Jinja2	inheritance, blocks, filters, loops, autoescaping
8	Blog CRUD: views, forms and templates	create, read, update and delete with validation and redirects
9	Making the application installable	`pyproject.toml`, editable installs, packaging data files
10	Testing Flask applications with pytest	fixtures, the test client, testing auth and the database
11	Deploying to production and next steps	WSGI servers, reverse proxies, secrets, extensions
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Hello, Flask

1.1 Summary

- Flask is a Python web framework that maps URLs to Python functions and turns the returned values into HTTP responses.
- It is a microframework: routing, requests, responses, sessions and templates are included, while databases, forms and authentication come from libraries.
- Flask sits on Werkzeug, which implements the WSGI protocol, and Jinja2, which renders templates.
- A WSGI application is a callable that receives an environment dictionary and a `start_response` callable, and returns an iterable of bytes.
- A view function reads data from the `request` object and returns a string, a tuple, a `Response` object or a rendered template.
- The route decorator registers the pairing between a URL rule and a view, so HTTP requests reach the right function.
- The development server restarts the application on every save and shows a debugger, which makes it convenient but unfit for production.

- Everything in this volume follows the official Flask tutorial path, extended with the pieces the tutorial leaves out, such as SQLAlchemy and WTForms.

1.2 Concepts

What a microframework means in practice

Flask has opinions about the request cycle and stays quiet about everything else. The table below shows the split between what you install with Flask and what you decide on your own.

Concern	Shipped with Flask	Usual choice for it
Routing	`Flask.route`, converters	-
Request and response	`request`, `Response`, `jsonify`	-
Templates	Jinja2 integration	-
Sessions	signed cookies	-
Static files	`/static` folder, cache busting	-
Database	-	SQLite, SQLAlchemy, PostgreSQL
Forms	-	hand-written validation, WTForms
Authentication	-	session recipe, Flask-Login
Migrations	-	Flask-Migrate, Alembic
Production serving	development server only	gunicorn, Waitress

The request cycle

Every request follows the same path. A WSGI server parses the socket into an environment dictionary, Werkzeug matches the URL against the rule map, Flask calls the matching view with the values captured from the URL, and the return value is converted into a response.

output

```
browser asks for GET /posts/3
            |
            v
+-----------------------+
| WSGI server (Werkzeug) |  builds the environ dict
+-----------------------+
            |
            v
+-----------------------+      rules: /, /posts/<int:post_id>
| URL map               |  ->  match: view post_detail, post_id=3
+-----------------------+
            |
            v
+-----------------------+      reads request.form, session, g
| view function         |      returns a Response
+-----------------------+
            |
            v
+-----------------------+
| status, headers, body  |  sent back through the same server
+-----------------------+
```

The Python you need before starting

Flask is deliberately plain Python, and a handful of language features appear everywhere.

Feature	Where it shows up
Decorators	`@app.route`, `@login_required`, `@app.errorhandler`
Closures	the route decorator registers a view and returns it
`__name__`	tells Flask where the package root and templates live
Keyword arguments	configuration keys and view values
Context managers	`with get_db()`, `with app.test_client()`
Type hints, optional	`def create_app() -> Flask`

1.3 Examples

Example 1: the request cycle without a framework, executed with Python 3.13.

Python

```python
"""Two tiny views written in plain Python, to show what a framework adds."""
from wsgiref.simple_server import make_server  # stdlib reference WSGI server


class Request:
    """A minimal stand-in for the request Werkzeug builds from the environment."""

    def __init__(self, method, path):
        self.method = method
        self.path = path


def hello(request):
    return "Hello, Flask reader!"


def about(request):
    return "Volume 25, Chapter 1."


ROUTES = {("GET", "/"): hello, ("GET", "/about"): about}


def dispatch(method, path):
    """Same idea as the Werkzeug map: method + path -> view, or 404."""
    view = ROUTES.get((method, path))
    if view is None:
        return 404, "Not found"
    return 200, view(None)


def application(environ, start_response):
    """A WSGI callable: the shape every Python web server understands."""
    status, body = dispatch(environ["REQUEST_METHOD"], environ["PATH_INFO"])
    start_response("%d OK" % status, [("Content-Type", "text/plain; charset=utf-8")])
    return [body.encode("utf-8")]


if __name__ == "__main__":
    for method, path in [("GET", "/"), ("GET", "/about"), ("GET", "/missing")]:
        print("%s %-9s -> %s" % (method, path, dispatch(method, path)))
    print("a server would run: make_server('127.0.0.1', 8000, application)")
```

output

```
GET /         -> (200, 'Hello, Flask reader!')
GET /about    -> (200, 'Volume 25, Chapter 1.')
GET /missing  -> (404, 'Not found')
a server would run: make_server('127.0.0.1', 8000, application)
```

Example 2: the same two views with Flask 3.1 (hand-checked, not executed).

Python

```python
from flask import Flask

app = Flask(__name__)


@app.get("/")
def index():
    return "Hello, Flask reader!"


@app.get("/about")
def about():
    return "Volume 25, Chapter 1."


if __name__ == "__main__":
    app.run(debug=True)
```

output

```
 * Serving Flask app 'hello'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit

# opening http://127.0.0.1:5000/ in a browser shows:
Hello, Flask reader!
```

`app.get` is a shortcut added in Flask 2.0 for `@app.route("/", methods=["GET"])`. The `__name__` argument tells Flask where the module lives, so it can find the `templates` and `static` folders relative to it.

Example 3: what the route decorator does, executed with Python 3.13.

Python

```
"""A decorator factory that records routes, which is what @app.route does."""
ROUTES = []


def route(rule, methods=("GET",)):
    """Return a decorator that stores the rule and returns the view unchanged."""

    def register(view):
        ROUTES.append((rule, tuple(methods), view.__name__))
        return view

    return register


@route("/", methods=("GET", "HEAD"))
def index():
    return "home"


@route("/posts/<int:post_id>", methods=("GET",))
def post_detail(post_id=None):
    return "post %s" % post_id


if __name__ == "__main__":
    for rule, methods, name in ROUTES:
        print("%-24s %-18s -> %s" % (rule, methods, name))
```

output

```
/                        ('GET', 'HEAD')    -> index
/posts/<int:post_id>     ('GET',)           -> post_detail
```

The decorator itself does not change the view. It records the rule in a table and hands the function back, which is why the view keeps its name and can be called directly in a test.

Example 4: installing Flask, and proving it is there.

shell

```
python -m venv .venv
python -m pip install "Flask>=3.1,<4"
python -c "import flask; print(flask.__version__)"
```

shell

```
$ python -c "import flask"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import flask
ModuleNotFoundError: No module named 'flask'
```

The last block is the real result on the writing machine, where Flask is not installed and no network was available. On a machine with Flask 3.1 installed, the same command prints the version, for example `3.1.2`.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Running `app.run()` in production	the Werkzeug server is single-process and not audited	use gunicorn or Waitress behind a proxy
Reading `os.environ` inside a view	values are frozen at import time and tests cannot override them	read configuration once, in the factory
Putting all views in one growing file	nothing can be tested or reused in isolation	group views into blueprints per feature
Using `debug=True` with a public host	the debugger allows arbitrary code execution	keep debug local, set it from configuration
Forgetting that `__name__` fixes the root path	templates and static files are not found	always pass `__name__` when constructing `Flask`
Comparing Flask examples from old blog posts	Flask 0.x and 1.x APIs were removed in 2.x and 3.x	read the current stable docs and pin `Flask>=3.1,<4`

1.5 Practice exercises

1. Name the two libraries Flask is built on and say what each one does.
2. Write a view that answers `GET /ping` with the text `pong`.
3. Two views have the same function name in the same module. What does Flask do, and how do you avoid the problem?
4. Explain why the development server prints a warning when it starts.
5. Which Python feature lets `@app.route("/x")` decorate a function without changing it?

Solutions

1. Werkzeug, which implements the WSGI protocol, routing and the request and response objects, and Jinja2, which renders templates.
2. `@app.get("/ping")` above `def ping(): return "pong"`.
3. Flask uses the function name as the endpoint, so the second rule replaces the first in the rule map; give each view a unique name or pass an explicit endpoint, for example `@app.get("/a", endpoint="index_a")`.
4. Because it is a development-only server: it is not designed for concurrency, security or performance, so it must never face the internet.
5. A decorator factory: `route` returns a decorator that records the rule, calls the view and returns it unchanged.

1.6 Mini test

1. What does a WSGI application receive from the server? A) A single string B) An environment dictionary and a `start_response` callable C) A `Flask` object D) A parsed `request` object

2. Which library renders templates in Flask? A) Mako B) Jinja2 C) Moustache D) Django templates
3. True or false: a view function must return a `Response` object.
4. What is the endpoint of a view by default?
5. Which command installs Flask 3.1 into the active virtual environment? A) `python install flask` B) `python -m pip install "Flask>=3.1,<4"` C) `flask install` D) `python -m venv Flask>=3.1`
6. Why does `Flask(__name__)` matter?
7. Which shortcut did Flask 2.0 add for GET-only routes? A) `@app.view("/x")` B) `@app.get("/x")` C) `@app.route_get("/x")` D) `@app.map("/x")`

Answer key and explanations

1. B - WSGI passes the environment and the response-starter to the application and expects an iterable of bytes back.
2. B - Jinja2 is bundled through Flask and used by `render_template`.
3. False - a plain string, a tuple of body and status, or a `Response` are all accepted and converted.
4. The name of the view function, unless an explicit `endpoint` is given.
5. B - `pip install` with a version range keeps the project on Flask 3.x.
6. It gives Flask the import path of the module, so it can locate the `templates` and `static` folders next to it.
7. B - `@app.get` is shorthand for `@app.route(..., methods=["GET"])`.

Chapter 2 - Project layout and the application factory

2.1 Summary

- A Flask application of any size is a Python package, and the tutorial layout keeps every feature module inside it.
- The application factory is a function that builds, configures and returns the `Flask` object.
- A factory makes testing possible, because every test can ask for a fresh app with its own configuration.
- The package root is fixed by `Flask(__name__)`, which is how the template and static folders are found.
- Blueprints are registered inside the factory, so nothing is imported before the app exists and circular imports disappear.
- The instance folder, outside the package, holds files that change at runtime, such as the SQLite database.
- `flask --app blogsmith run` finds the application by name and starts the development server.

- A module-level `app` global still works for a tiny script, but it does not scale to tests, several configurations or a team.

2.2 Concepts

Why a factory instead of a global app

Writing `app = Flask(__name__)` at module level is the fastest way to start and the hardest way to maintain. The factory moves construction into a function that can be called as many times as you like.

Approach	When the app is built	Two configurations	Tests
module-level `app`	at import time	edit the module	all tests share one app and one database
`create_app()`	when called	`create_app("testing")`	one app per test session

The layout

output

```
blogsmith/
├── pyproject.toml
├── README.md
├── src/
│   └── blogsmith/
│       ├── __init__.py       # create_app(), the factory
│       ├── db.py             # open_connection(), close_connection(), init_db()
│       ├── schema.sql        # CREATE TABLE statements
│       ├── auth.py           # Blueprint("auth", __name__, url_prefix="/auth")
│       ├── blog.py           # Blueprint("blog", __name__, url_prefix="/blog")
│       ├── cli.py            # optional console entry point
│       ├── static/
│       │   └── style.css
│       └── templates/
│           ├── base.html
│           ├── auth/
│           │   ├── register.html
│           │   └── login.html
│           └── blog/
│               ├── index.html
│               ├── detail.html
│               ├── create.html
│               └── update.html
└── tests/
    ├── conftest.py
    ├── test_auth.py
    ├── test_blog.py
    └── test_db.py
```

The `src/` directory is not required, but it stops the package from being imported accidentally while you work in the project root. The instance folder is not in the tree: Flask creates it next to the package the first time something needs it, and it stores `blogsmith.sqlite` and any local `config.py`.

How Flask finds the application

The `--app` option accepts three shapes.

Value	What Flask looks for
`blogsmith`	`create_app` or `app` in the package
`blogsmith:app`	the `app` attribute, factory detection skipped
`blogsmith:create_app()`	a factory call, arguments allowed

The `FLASK_APP` environment variable does the same job when you do not want to repeat the flag. Since Flask 3.0 the `FLASK_ENV` and `FLASK_DEBUG` variables were removed: debug mode is enabled with the `--debug` flag, or with `app.run(debug=True)` in code.

2.3 Examples

Example 1: the factory, with the imports kept at the bottom on purpose.

Python

```python
"""src/blogsmith/__init__.py"""
import os

from flask import Flask


def create_app(test_config=None):
    """Create the application: configuration, then database, then blueprints."""
    app = Flask(__name__, instance_relative_config=True)

    app.config.from_mapping(
        SECRET_KEY="dev",
        DATABASE=os.path.join(app.instance_path, "blogsmith.sqlite"),
    )

    if test_config is None:
        # An optional file in the instance folder; missing is not an error.
        app.config.from_pyfile("config.py", silent=True)
    else:
        # Tests pass a dictionary, for example {"DATABASE": ":memory:"}.
        app.config.from_mapping(test_config)

    from . import db

    db.init_app(app)

    from . import auth, blog

    app.register_blueprint(auth.bp)
    app.register_blueprint(blog.bp)

    @app.route("/healthz")
    def healthz():
        return {"status": "ok"}

    return app
```

The imports sit inside the function on purpose. Each blueprint imports names from the package at its own import time, so deferring the imports keeps the import graph acyclic: the app is built first, then the blueprints register.

Example 2: the same idea in miniature, executed with Python 3.13.

Python

```python
from packagedemo import __version__, create_app

routes = create_app()
print("version:", __version__)
for path, view in routes.items():
    print("%-5s -> %s" % (path, view()))
```

output

```
version: 1.0.0
/     -> Hello, world!
/bye  -> Goodbye, world.
```

The package `__init__.py` re-exports the two views and defines the factory, so an importer never needs to know which module holds what. Flask packages follow the same shape: `create_app` is the public door.

Example 3: returning a dictionary from a view, hand-checked Flask 3.1 behaviour.

Python

```python
@app.get("/healthz")
def healthz():
    return {"status": "ok"}
```

output

```
# GET /healthz answers with one JSON document:
{"status":"ok"}
```

Since Flask 2.2 a dictionary returned by a view is serialised as JSON with a `application/json` content type. `jsonify` remains available for lists and for explicit status codes.

Example 4: starting the development server, expected output.

shell

```
$ flask --app blogsmith run
 * Serving Flask app 'blogsmith'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Add `--debug` while developing: the server reloads when a file changes and shows a traceback in the browser when a view raises. Never leave it on a public host, because the debugger executes code.

Example 5: listing the rule map, expected output.

shell

```
flask --app blogsmith routes
```

output

```
Endpoint      Methods   Rule
------------- --------- -----------------------
healthz       GET       /healthz
static        GET       /static/<path:filename>
index         GET       /
auth.register GET, POST /auth/register
auth.login    GET, POST /auth/login
auth.logout   GET       /auth/logout
blog.index    GET       /blog
blog.create   GET, POST /blog/create
```

Werkzeug pads the columns, so the widths change with the endpoint names. The endpoint name is what `url_for` expects: a view `index` inside the blueprint `blog` is called `blog.index`.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Building the app at module level	one configuration for every test and worker	use `create_app()` and pass settings in
Importing blueprints at the top of `__init__.py`	circular imports, half-built app	import them inside the factory, after the app exists
Storing the database inside the package	files in the package are replaced on upgrade	put it in the instance folder
Relying on `FLASK_ENV` or `FLASK_DEBUG`	both were removed in Flask 3.0	use `flask --debug run`
Forgetting `src/` or the install step	`blogsmith` is not importable, `flask --app` fails	install the package, or set `PYTHONPATH` once
Hard-coding secrets in the package	the secret ends up in version control	read them from the environment

2.5 Practice exercises

1. Why is a factory better than a module-level `app` for a project with tests?
2. Which two behaviours does `instance_relative_config=True` change?
3. Where does the SQLite file live, and why not inside the package?
4. What does `flask --app blogsmith:create_app()` ask Flask to do?
5. Which flag turns on debug mode for the development server, and why is it dangerous on a public host?

Solutions

1. A factory can be called once per test with a different configuration, so tests get an isolated application and an isolated database instead of sharing one global object.
2. Relative paths in the configuration, such as `from_pyfile("config.py")`, are resolved against the instance folder instead of the package root, and `app.instance_path` is created by default when the app needs it.
3. In the instance folder, which `app.instance_path` points to. Files inside the package are part of the distribution and are replaced or ignored when the package is reinstalled.
4. Import the `blogsmith` package, then call the `create_app()` factory; the object it returns is used as the application.
5. `--debug`, for example `flask --app blogsmith run --debug`. The debugger it enables lets anyone who can trigger an exception run Python code in the server process, so it must stay behind a local, protected interface.

2.6 Mini test

1. What does an application factory return? A) A module B) A configured `Flask` instance C) A blueprint D) A WSGI environment dictionary
2. Why does the factory import `db` and the blueprints inside the function? A) It saves memory B) The modules import from the package, so the imports must happen after the app object exists to avoid a cycle C) Python requires it D) It makes the routes case-insensitive
3. Which folder holds files that change at runtime? A) `src/blogsmith/` B) `tests/` C) the instance folder D) `static/`
4. True or false: `flask --app blogsmith run` needs a `create_app` function to be present.
5. Which endpoint name does `url_for` use for the view `index` in the blueprint `blog`? A) `index` B) `blog.index` C) `blog:index` D) `/blog/index`
6. What happens when `flask routes` is executed? A) The server starts B) Every registered rule is printed with its endpoint and methods C) The database is created D) The templates are compiled

Answer key and explanations

1. B - the factory is a normal function whose return value is the application object the server and the tests use.
2. B - a blueprint module does `from . import db` or similar, so importing it while the package is still initialising would find a half-built module.
3. C - the instance folder is writable, survives upgrades of the package and is outside version control by convention.
4. False - Flask looks for `create_app` or `app`; a plain `app` attribute is equally valid.
5. B - endpoints are namespaced by blueprint, written as `blueprint.view`.
6. B - the command reads the URL map and prints one line per rule.